NORTHERN UNIVERSITY OF BUSINESS & TECHNOLOGY KHULNA

CSE 4207: Pattern Recognition

Discussion On Hyperparameter Tuning

Section: 8B

Presented By

Md.Mahfujul Karim Sheikh
ID:11220120735

Md Riaz Ahmed
ID:11220120740

Suraiya Yeasmin
ID:11220120727

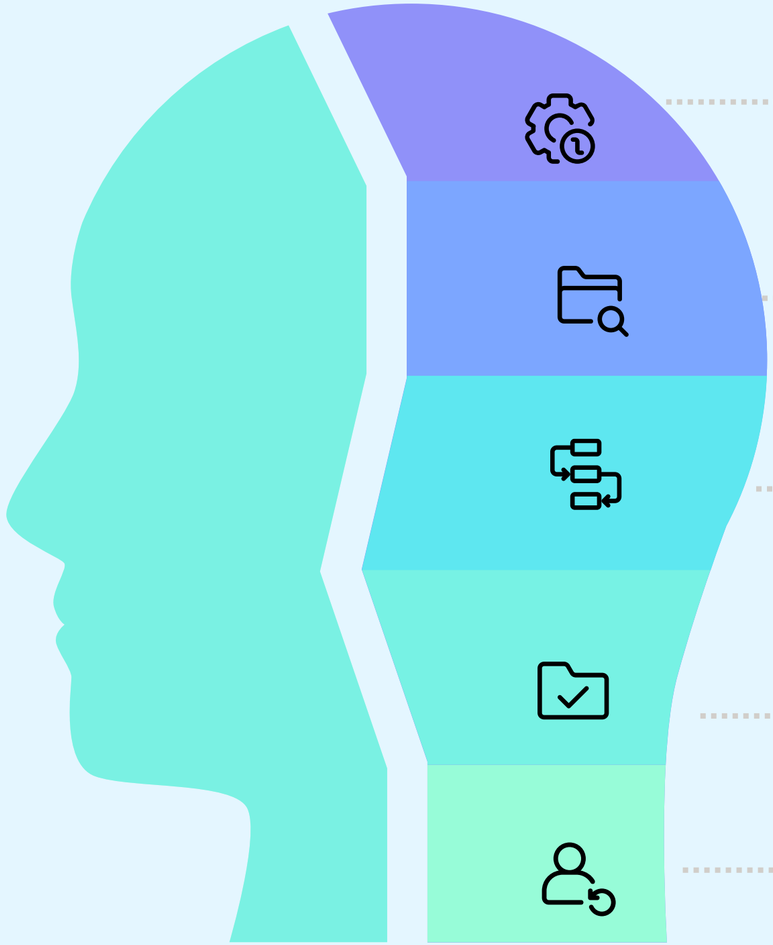
Dept. Of CSE, Northern University of Business And Technology Khulna

Presented To

Md. Azizul Islam
Senior Lecturer &
Coordinator

Dept. Of CSE, Northern University of Business And Technology Khulna

Today’s Agenda



01 Parameter & Hyperparameter

02 Hyperparameter Tuning Necessity

03 Hyperparameter Tuning Types
- 1.Grid Search
- 2.Randomized Search
- 3.Bayesian Optimization

04 Advantage , Disadvantage and Limitation

05 Hands-On Experiment For MLOPs Pipeline

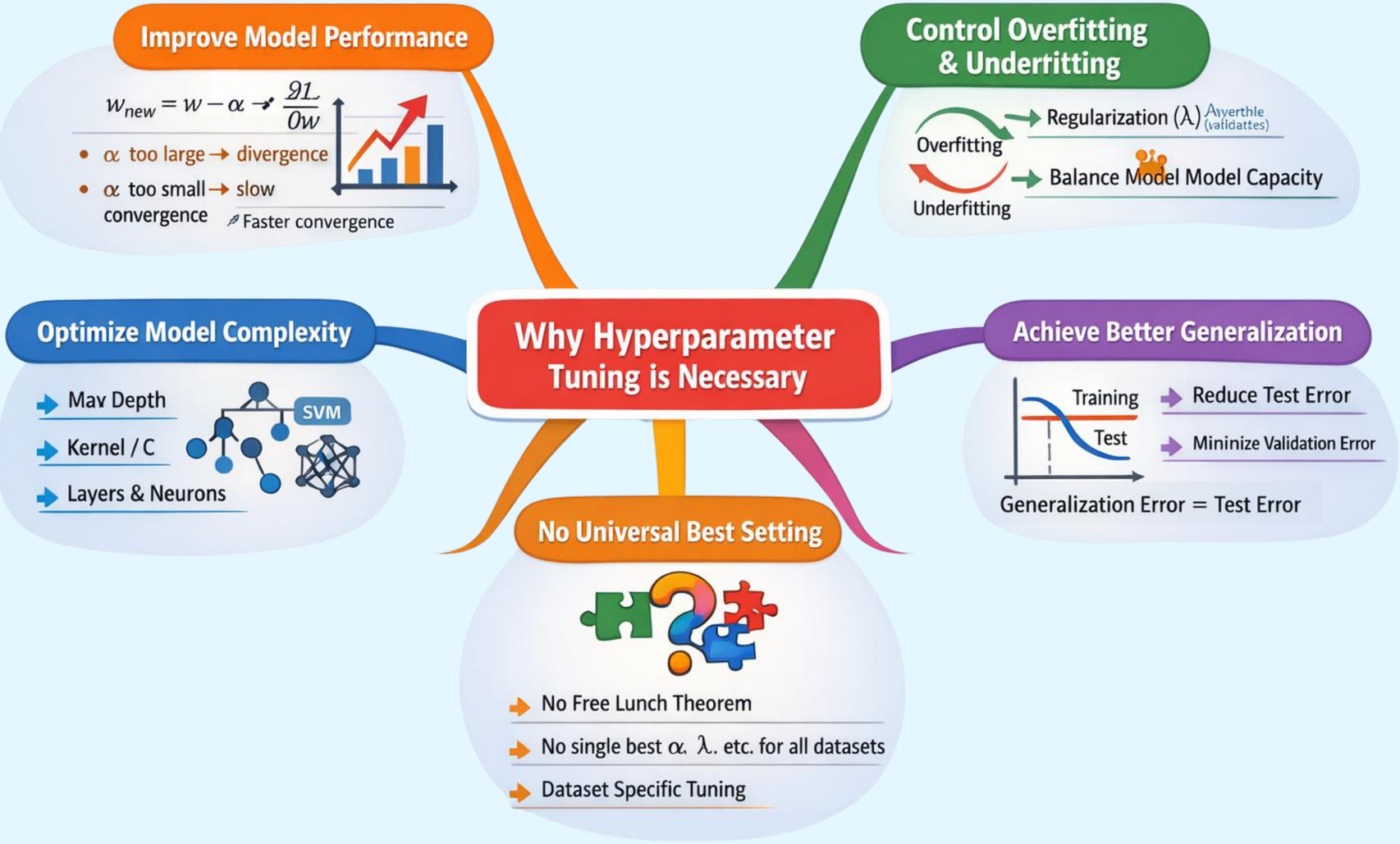
Parameter & Hyperparameter

Basically , there are two kinds of parameter.

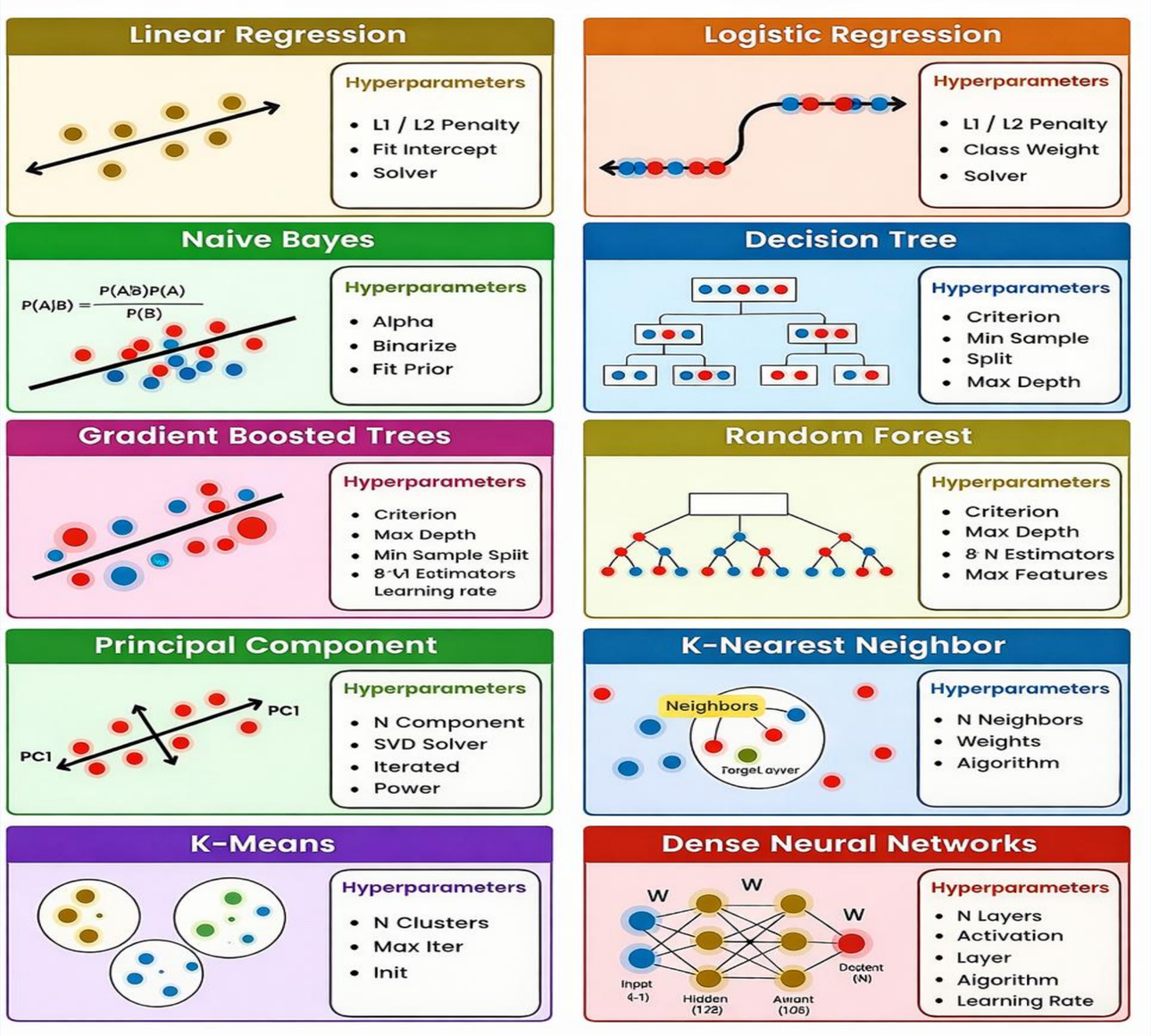
1. Model Parameter
2. Hyperparameter

Concept	Parameter	Hyperparameter (α)
Model / Update Equation	$\hat{y} = wx + b$	$w_{\text{new}} = w - \alpha \cdot \frac{\partial L}{\partial w}$
Symbol (example)	(w, b)	(α)
Where it appears	Inside model equation	Controls learning process
Learned automatically?	Yes ✓	No ✗
Mathematical role	$\theta^* = \text{argmin } L(\theta)$	$\lambda^* = \text{argmin ValLoss}$
Changes during training?	Yes	No (fixed during run)
Example values	$w = 2.5$	$\alpha = 0.01$

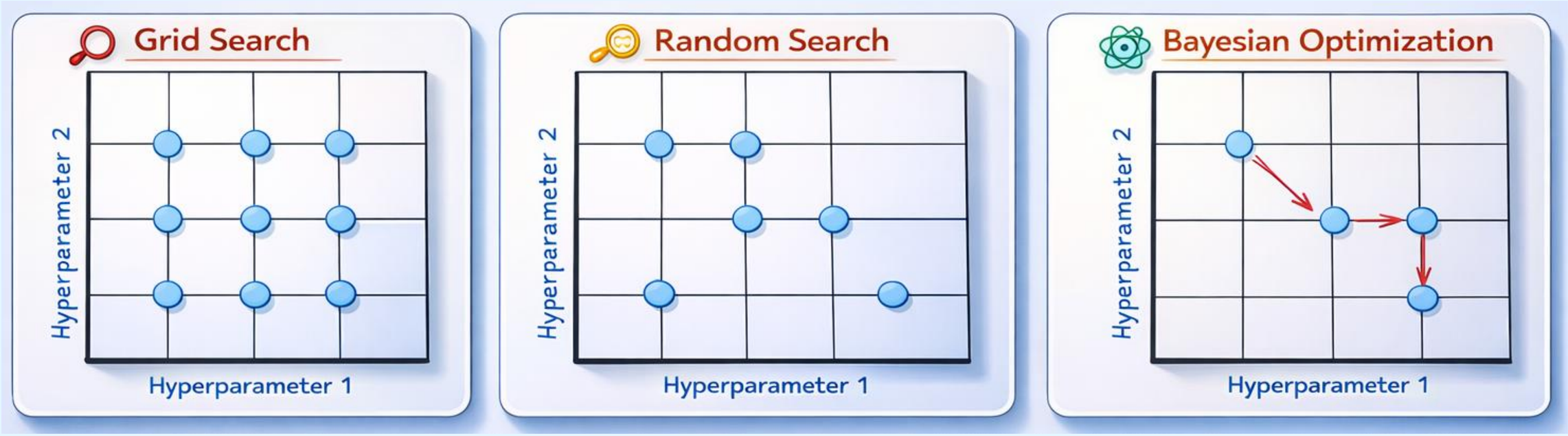
Hyperparameter Tuning Necessity



Hyperparameter of Traditional ML Model



Hyperparameter Tuning Types



4-Mar-26

GridSearch Implementation WorkFlow

1. Model (Hypothesis Function)

$$\hat{y} = wx + b$$

2. Cost Function (MSE)

$$J(w, b) = \frac{1}{2n}\sum(\hat{y} - y)^2$$

3. Gradients

$$\frac{\partial J}{\partial w} = \frac{1}{n}\sum(\hat{y} - y)x$$

$$\frac{\partial J}{\partial b} = \frac{1}{n}\sum(\hat{y} - y)$$

4. Update Rule (Gradient Descent)

$$w := w - \alpha\frac{1}{n}\sum(\hat{y} - y)x$$

$$b := b - \alpha\frac{1}{n}\sum(\hat{y} - y)$$

Problem Statement

Step1: x=0, y=0

Step2: [−2,−4,−6,−8]

Step3: $\frac{\partial J}{\partial w} - 15$

$$\frac{\partial J}{\partial b} = -5$$

Step4:
α∈{0.01, 0.1, 0.5}

- **α = 0.01**

$$w = 0.15, b = 0.05$$

- **α = 0.1**

$$w = 1.5, b = 0.5$$

- **α = 0.5**

$$w = 7.5, b = 2.5$$

Best Learning Rate: 0.1

Final Model: y=1.5x+0.5

Variable (x)	Result (y)
1	2
2	4
3	6
4	8

Actual Model: y=2x

Randomized Search Algorithm

Goal: Find the best hyperparameters by randomly sampling combinations.

Algorithm Steps

1 Define Hyperparameter Distributions

- Learning rate ~ Uniform(0.001, 0.1)
- Max depth ∈ {3, 5, 7, 9}

2 Choose Number of Iterations (N)

Decide how many random combinations to test.

3 For *i* = 1 to N:

- Randomly sample a hyperparameter combination
- *Train the model using sampled parameters*
- Evaluate using validation score

4 Store Results

Save (hyperparameters, score)

5 Select Best Combination

Choose parameters with highest validation performance

Output:

Best hyperparameter set after N random trials.

Key Idea:

- *Instead of testing **all combinations** (Grid Search),*
- *Randomized Search tests **random** combinations,*
  - → *Faster*
  - → *Efficient for large search spaces*

🎯 Bayesian Optimization Algorithm

(Hyperparameter Tuning)

Goal: Find the best hyperparameters using a probabilistic model to guide the search.

✅ Algorithm Steps

1 Define Search Space

- Example:
  - Learning rate ∈ [0.001, 0.1]
  - Max depth ∈ {3, 5, 7, 9}

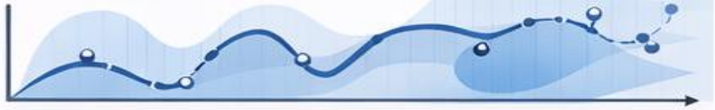
2 Initialize with Few Random Points

- Train model on 3–5 random hyperparameter sets
- Record validation scores

3 Build Surrogate Model

- Fit a probabilistic model (e.g. Gaussian Process)

$f(x) \approx$ **Validation Score**



3 Build Surrogate Model

- Fit a probabilistic model (e.g. Gaussian Process)
- Approximates objective function:

$$f(x) \approx \text{Validation Score}$$

4 Use Acquisition Function

- Decide next best hyperparameter to test
- Balance:

🔍 Exploration (new areas)

🎯 Exploitation (best areas)

5 Evaluate & Update

- Train model with selected parameters
- Update surrogate model with new result

6 Repeat Until Budget Ends

🏆 Output:

Best hyperparameter set found during iterations.

💡 Key Idea:

- Unlike Grid or Random Search,
  Bayesian Optimization *learns from previous trials*
  → *Requires fewer evaluations*
  → *More efficient for expensive models*

Advantage & Disadvantage

Advantages

1. Improves Model Performance

Finds optimal configuration that minimizes loss

Increases accuracy, precision, recall: F1-score

Example: Proper C improves convergence in Gradient Descent

2. Reduces Overfitting & Underfitting

Controls model complestly

Example: Large depth → Overfitting

Small depth → Underfitting

3. Better Generalization

Improves vatdation/test performance

Produces more robust models

4. Optimizes Training Elficiency

Correct learning rate → Faster convergence

Avoids unstable training

5. Required for Many ML Models

Essential for modets like:

- ✔ Support Vector Machine
- ✔ Randem Forest
- ✔ Neurel Network

Disadvantages

1. Computationally Expensive

Grid Search requires training model multiple times

Total Models = $k_1 \times k_2 \times \ldots \times k_n$

2. Time-Consuming

Large search space → Long training time

Esperially costly for deep learning

3. Risk of Overfitting to Validation Set

Excessive Tuning may adapt too much to validation data

4. No Guarantee of Global Optimum

Grid search may miss bettor combination

Random search is stochastic

5. Requires Domain Knowledge

Selecting search range needs eyperionce

Pour range → Ineffeient luning

Hyperparameter Tuning Limitations

1. High Computational Cost

Tetal Models = $k_1 \times k_2 \times \cdots \times k_n$

Deep Neural Nelworks

Large Datasets

Cross Validation (k Fold)

2. Time-Consuming

Train + Valldate

100 x 10 = 1000 Minutes

Slow Process

3. Curse of Dimensionalny

Search Space oc $|T|^{|H|}$

- Expioding Starch Space
- Inefficienn Evotoration

4. Overfitting Risk

Modd Learns Vallaation Noise

Seleclion Bias

4. Overfitting Risk

Selection Blas

Poor Generalıeation

7. Implementation Complesity

Bqueslan Oprfuinatıom

Hyperband

Parallel Compining

Evoluliorore Algrıthms

5: No Global Optımum

$0^{\theta} = \arg\min_{\theta} 0_{\theta}(1/9)$

- Non-Conves Function
- Nolsy Evaluation

6. Resource Intensive

High RAVI

CPU / TPU

Paralel Compating

8. Diminishing Returns

Δ Performance ≈ 0

Minimal Improvement

Hands-On Experiment

[Mahfujul-01726/Hyperparameter-Tuning]

Conclusion

•Hyperparameter tuning is a **critical step** in machine learning model optimization.

•It directly impacts **accuracy, generalization, convergence speed, and model stability**.

•Techniques like:

•**Grid Search** → systematic but computationally expensive

•**Randomized Search** → faster and more efficient for large search spaces

•**Bayesian Optimization** → intelligent and performance-driven search

•Proper tuning helps:

•Reduce **overfitting and underfitting**

•Improve **validation & test performance**

•Build **robust and production-ready models**

Questions & Answers